

Basic Information

- **Title:** UPI Payment System using Interface and Custom Exception Handling

Objective / Aim

To write a Java program that demonstrates the use of **interfaces, implementation, custom exceptions, exception handling, and the finally block** by developing a simple UPI payment system.

System Requirements

Hardware

- Standard PC/Laptop
- Core i3/i5 Processor or equivalent
- 4 GB/8 GB RAM

Software

- Windows/Linux/macOS
- JDK 17 or JDK 21
- VS Code / Eclipse / IntelliJ IDEA / NetBeans
- Command Prompt / Terminal

Algorithm / Logic Description

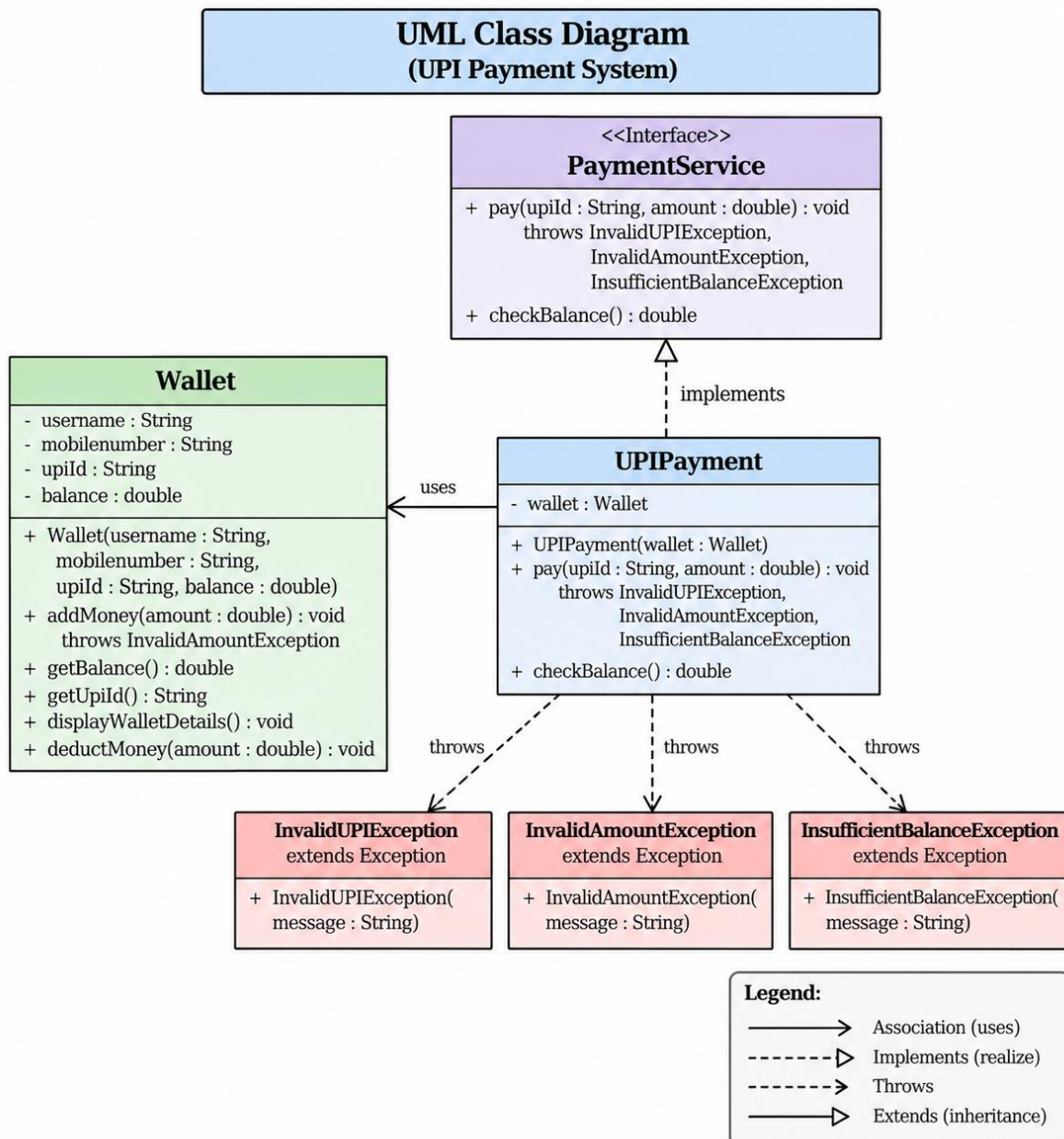
1. Start the program.
2. Create an interface named PaymentService.
3. Declare pay() and checkBalance() methods in the interface.
4. Create a Wallet class to store username, mobile number, UPI ID, and balance.
5. Define methods to add money, check balance, display wallet details, and deduct money.
6. Create three custom exceptions:
 - InvalidUPIException
 - InvalidAmountException
 - InsufficientBalanceException
7. Create the UPIPayment class that implements PaymentService.
8. Override the pay() method.

9. Validate the UPI ID using the @ symbol.
10. Validate whether the payment amount is greater than zero.
11. Check whether the wallet has sufficient balance.
12. Throw the appropriate custom exception if validation fails.
13. Deduct the payment amount from the wallet when the payment is valid.
14. Create a Wallet object in the main() method.
15. Add money to the wallet.
16. Perform a UPI payment.
17. Handle exceptions using try-catch.
18. Execute the finally block to display that the transaction is completed.
19. Display the final wallet balance.
20. Stop the program.

UML Class Diagram

Here is the UML diagram for **your exact program structure**:

Main relationships to show



Source Code

Your source code is based on these main concepts:

- **Interface:** PaymentService
- **Implementation:** UPIPayment implements PaymentService
- **Class:** Wallet
- **Custom Exceptions:** Three exception classes
- **Method Overriding:** pay() and checkBalance()
- **Exception Handling:** try-catch-finally
- **Encapsulation:** Wallet variables are private

Your existing code is suitable for this experiment.

Important concepts

Concept	Your Code
Interface	PaymentService
Implements	UPIPayment implements PaymentService
Custom Exception	InvalidUPIException
Custom Exception	InvalidAmountException
Custom Exception	InsufficientBalanceException
Exception Inheritance extends Exception	
Method Overriding	@Override pay()
Try-Catch	Handles exceptions
Finally	Transaction completion
Encapsulation	private Wallet variables
Constructor	Wallet(...), UPIPayment(...)

Compilation & Execution Commands

Your public class is:

UpiPayementSystem

So save the file exactly as:

UpiPayementSystem.java

Compile

```
javac UpiPayementSystem.java
```

Run

```
java UpiPayementSystem
```

Note: Your class name is spelled UpiPayementSystem in the code (Payement), not UpiPaymentSystem. The filename must match it exactly.

Output Screen

For your current input values, the output will be:

Wallet Details

User Name: Sai

Mobile Number: 90000000000

UPI ID: sai@upi

Balance: 50000.0

2000.0 added to wallet

Payment successful

Amount Paid: 15000.0

Paid to: riya@upi

Available Balance: 37000.0

Transaction completed

Wallet Details

User Name: Sai

Mobile Number: 90000000000

UPI ID: sai@upi

Balance: 37000.0

Final Wallet Balance: 37000.0

Result

Thus, the Java program for the UPI Payment System was successfully implemented and executed using an interface, custom exception handling, method overriding, encapsulation, and try-catch-finally blocks.